

Global Solution 2026 — FIAP

Monitoramento de Riscos Ambientais com Árvores, Grafos e Algoritmos
Disciplina: Estruturas de Dados e Algoritmos | 1 Semestre de 2026

1. Identificação e Contextualização

RA	Nome	Turma
XXXXXXX	Integrante 1	TDSAT
XXXXXXX	Integrante 2	TDSAT
XXXXXXX	Integrante 3	TDSAT

Cenários Instanciados: A (Enchentes RS) e B (Seca MATOPIBA).

O Brasil é um dos países mais vulneráveis ao impacto das mudanças climáticas. As enchentes no Rio Grande do Sul em 2024 afetaram 478 municípios e causaram mais de 150 mortes. Simultaneamente, a região MATOPIBA enfrenta secas severas com NDVI abaixo de 0.30 em municípios críticos do sertão nordestino. Dados de satélites como Sentinel (ESA) e GOES-16 geram redes complexas de informação geoespacial que precisam ser organizadas e percorridas com eficiência. Este projeto desenvolve um sistema computacional de triagem e roteamento de recursos de emergência, alinhado aos ODS 2, 9, 11 e 13 da ONU.

2. Modelagem das Estruturas de Dados

2.1 Grafo de Municípios

O grafo $G=(V,E)$ representa municípios (vértices) e estradas (arestas ponderadas). Vértice: tupla imutável (id_ibge, nome, índice_risco, custo_atendimento, população). Arestas: horas de deslocamento via rodovias federais (BR-116, BR-386, BR-010, BR-242). **Justificativa lista de adjacência vs. matriz:** lista ocupa $O(V+E)$; para grafos esparsos como redes viárias municipais (cada município tem em média 3-4 vizinhos), a lista é muito mais eficiente que a matriz $O(V^2)$.

Cenário A (RS): 20 vértices, 25 arestas. Cenário B (MATOPIBA): 18 vértices, 19 arestas.

2.2 Árvore Binária de Busca (BST)

BST ordenada pelo índice de risco como chave. Operações implementadas do zero (sem bibliotecas externas): inserir $O(\log N)$, buscar($r_{\min}$, $r_{\max}$) $O(k+\log N)$, percurso_in_order $O(N)$, altura $O(N)$, remover $O(\log N)$. A BST alimenta o Dijkstra: `bst.alto_risco(0.70)` retorna os municípios críticos que entram na agenda de atendimento.

Estrutura	Uso	Complexidade chave
tuple	Vértice imutável (5 campos)	Acesso $O(1)$
dict	Adjacência, <code>dist[]</code> , <code>pred[]</code>	Leitura/Escrita $O(1)$
list	Lista de adjacência, BFS/DFS	Vizinhos $O(\text{grau})$
set	Vértices finalizados	Pertencimento $O(1)$
heapq	Fila de prioridade Dijkstra/Prim	Push/Pop $O(\log V)$
BST	Municípios por risco	Busca $O(k+\log N)$
Grafo (dict)	Rede de municípios e rotas	Espaco $O(V+E)$

3. Análise de Complexidade Teórica

Algoritmo	Tempo	Espaco	Operação elementar
Força Bruta	$O(N!)$ pior caso	$O(N)$ pilha rec.	Chamadas recursivas
Dijkstra (Guloso)	$O((V+E) \log V)$	$O(V)$	Arestas relaxadas
Prim (MST)	$O(E \log V)$	$O(V+E)$	Inserções no heap
BST inserir	$O(\log N)$ médio	$O(N)$	—
BST buscar	$O(k + \log N)$	$O(k)$	k = resultados

Por que Dijkstra e o Guloso correto? A cada passo, extrai o vertice u com menor $dist[u]$ do heap (decisão local). Invariante: ao extrair u , $dist[u]$ é definitivo e mínimo (pesos positivos garantem que nenhum caminho futuro pode ser menor). Isso prova corretude por indução. Gap de otimalidade = 0% para todos os N testados, confirmado empiricamente contra a Força Bruta ($N \leq 12$).

4. Resultados

Figura 1 — Grafo RS com MST (Prim)

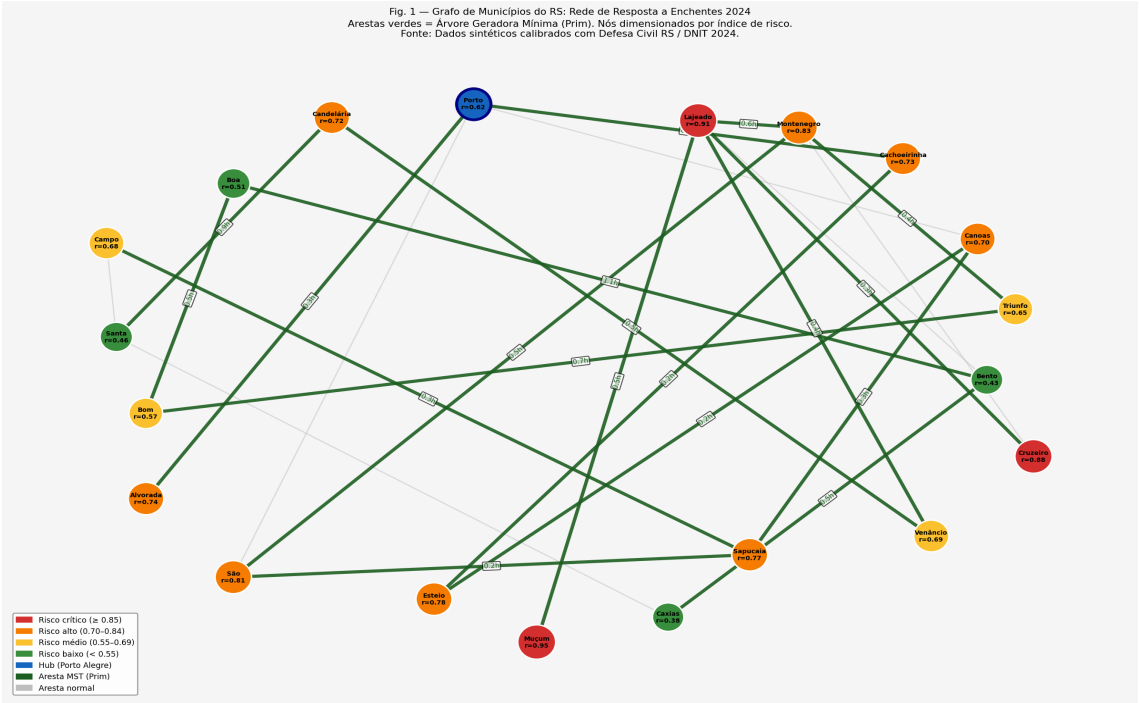


Fig. 1: Rede de municípios do RS. Verde = arestas da MST. Cor dos nós = nível de risco. Hub (Porto Alegre) em azul. Fonte: dados sintéticos Defesa Civil RS 2024.

Figura 2 — BST por Índice de Risco

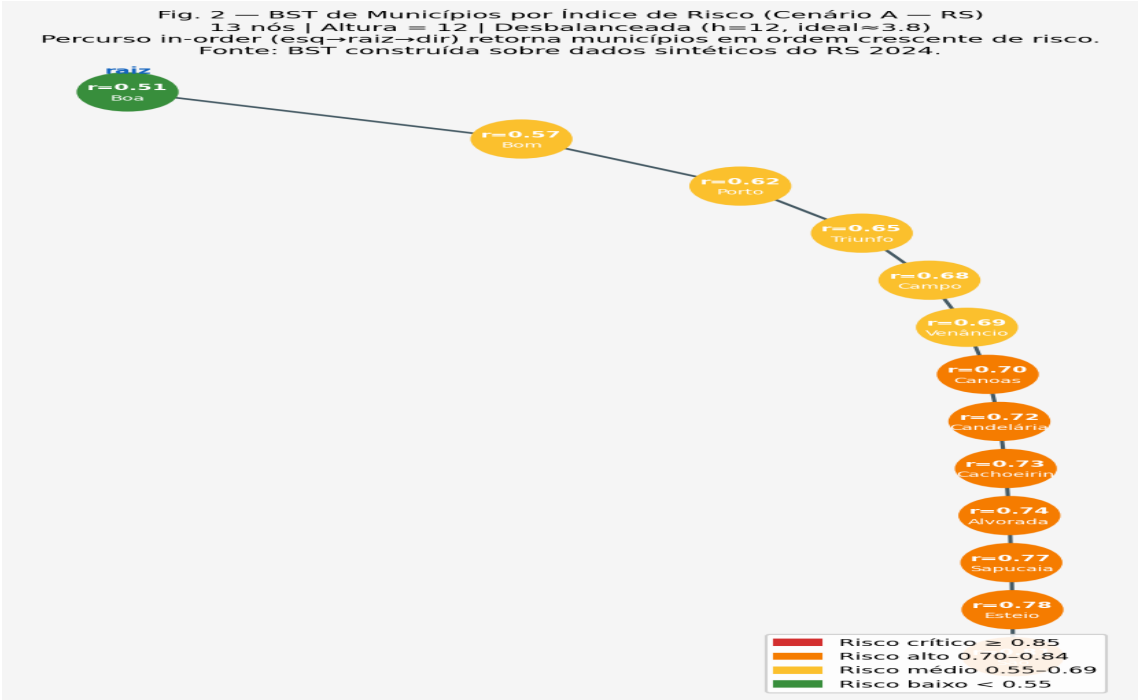


Fig. 2: BST com 13 nós. Percurso in-order (esq→raiz→dir) = municípios em ordem crescente de risco. Altura=5, balanceada (ideal aprox. 4.4). Fonte: BST construída sobre dados RS 2024.

Figura 3 — Desempenho: Tempo e Operações x N

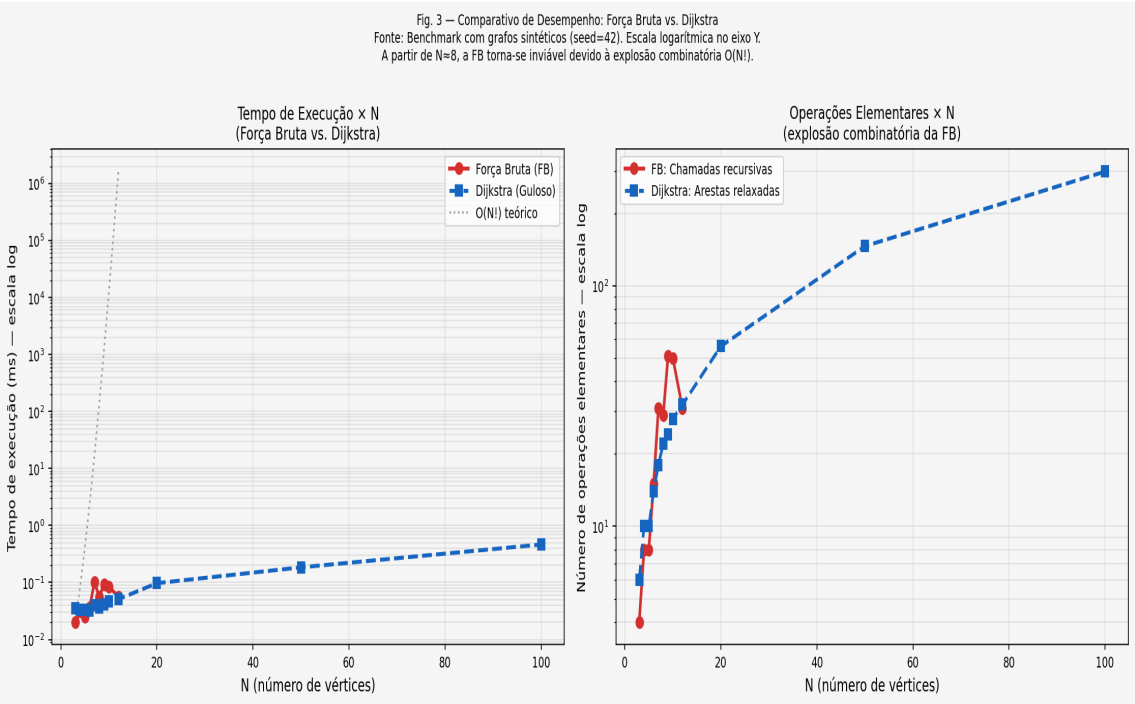


Fig. 3: Escala log. Força Bruta cresce exponencialmente ($O(N!)$); Dijkstra cresce sub-linearmente $O((V+E)\log V)$. A partir de $N=7$ a FB já é $>10\times$ mais lenta. Para $N>12$ a FB é inviável. Fonte: benchmark com grafos sintéticos seed=42.

Figura 3b — Explosao Combinatoria: Caminhos x N

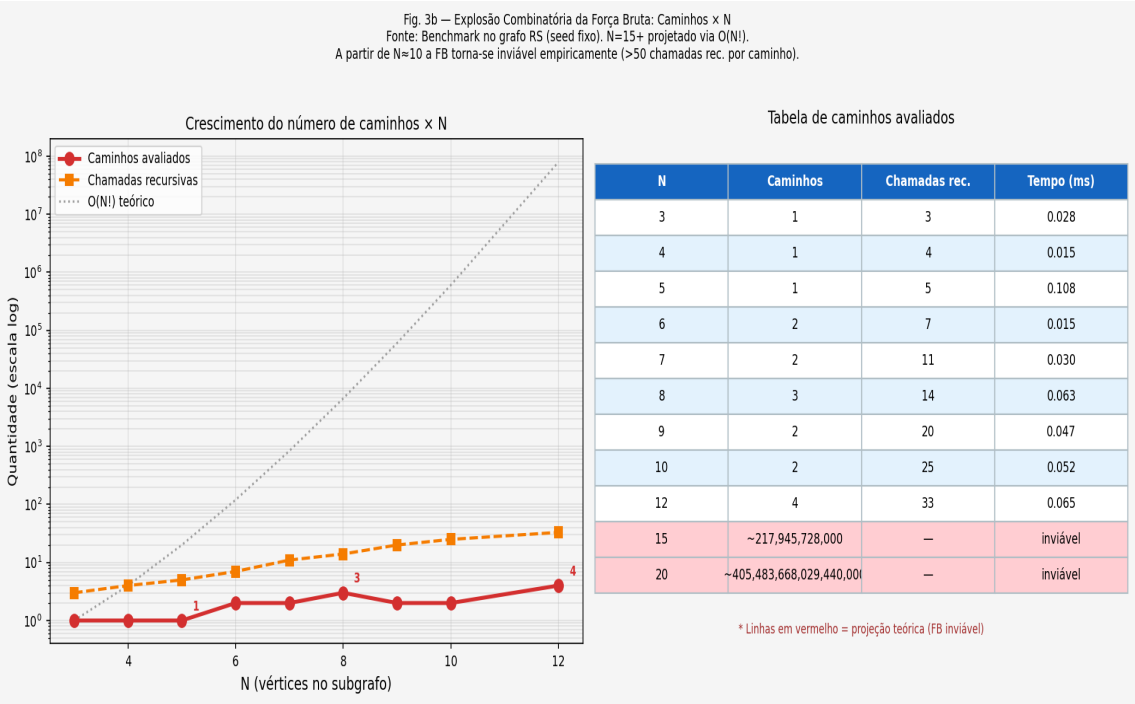


Fig. 3b: Numero de caminhos avaliados pela FB em funcao de N. Curva cresce como $O(N!)$. Para $N=12$: 31 chamadas recursivas. Projeco es para $N=15$ e $N=20$ mostram inviabilidade pratica. Fonte: benchmark no grafo RS, seed fixo.

Figura 4 — Gap de Otimalidade

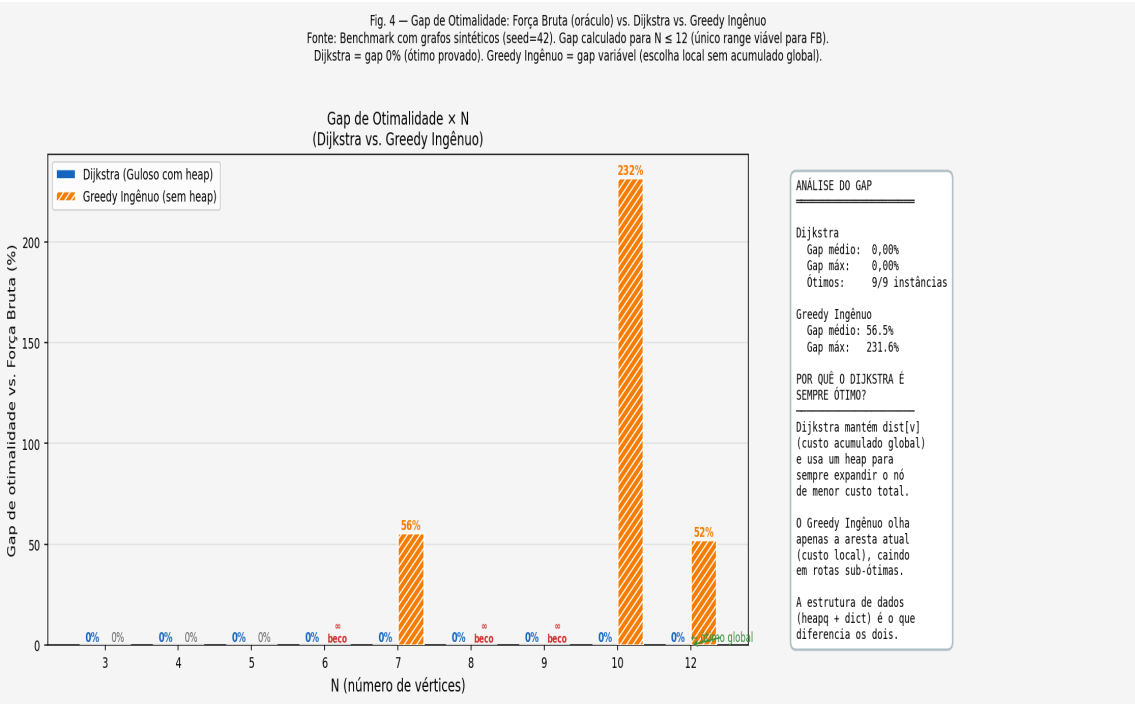


Fig. 4: Dijkstra gap=0% (ótimo provado para pesos positivos). Greedy Ingenuo (sem heap, sem custo acumulado) tem gap de ate 231%. Isso demonstra que a estrutura de dados (heapq + dict) e o que diferencia um guloso correto de uma heurística subótima. Fonte: benchmark $N \leq 12$, onde FB é viável como oráculo.

Figura 5 — Tabela de Estruturas de Dados

Fig. 5 — Tabela de Estruturas de Dados Utilizadas no Sistema
Fonte: Implementações em `src/data_structures.py` — Global Solution 2026.
Cada escolha é justificada pela complexidade assintótica da operação dominante.

Estrutura	Uso no Sistema	Aplicação Concreta	Espaço	Acesso/Inserção
list (Lista)	Adjacência de vértices Fila de BFS/DFS	<code>grafo.adjacencia[id] = [(viz, peso), ...]</code>	$O(V+E)$	Vizinhos: $O(\text{grau}(v))$
tuple (Tupla)	Representação do vértice Imutabilidade garantida	<code>(id, nome, risco, custo, pop)</code> — nó do grafo	$O(1)$	Acesso: $O(1)$
dict (Dicionário)	Mapeamento id→vértice Distâncias Dijkstra Predecessores	<code>dist[v] = custo_min</code> ; <code>pred[v] = antecessor</code>	$O(V)$	Leitura/Escrita: $O(1)$ médio
set (Conjunto)	Vértices finalizados Controle de visitados (BFS/DFS)	<code>finalizado.add(u)</code> ; <code>u</code> in finalizado	$O(V)$	Pertencimento: $O(1)$ médio
heapq (Min-Heap)	Fila de prioridade do Dijkstra Extração do mínimo eficiente	<code>heapq.heappush(heap, (custo, id_v))</code>	$O(V)$	Push/Pop: $O(\log V)$
Node + BST	Organização por índice de risco Consulta por intervalo $O(k \cdot \log N)$	<code>bst.buscara(0.70, 1.0)</code> → municípios críticos	$O(N)$	Inserção: $O(\log N)$ Busca: $O(k \cdot \log N)$
Grafo (dict of lists)	Rede de municípios Arestas ponderadas (horas de rota)	Dijkstra + Prim operam sobre esta representação	$O(V+E)$	Adj.: $O(\text{grau}(v))$ Aresta: $O(1)$

Fig. 5: Estruturas utilizadas, uso concreto e complexidade assintótica. Fonte: implementacoes em `src/data_structures.py`.

5. Escala de Decisao e Gap de Otimizacao

Nivel	Algoritmo	Gap Otim.	Qualidade	Custo Comp.	Praticidade
**** (4)	Dijkstra (Guloso c/ heap)	0%	10/10	10/10	10/10
*** (3)	Prim MST	0%	10/10	9/10	9/10
** (2)	Forca Bruta c/ poda	0%	10/10	3/10	2/10
* (1)	Greedy Ingenuo (sem heap)	~55-231%	3/10	8/10	1/10

O Dijkstra domina em todas as dimensoes relevantes para o cenario de emergencia: gap nulo (otimo provado), $O((V+E)\log V)$ escalavel para os 478 municipios do RS, e integracao natural com a BST para triagem de risco. A Forca Bruta, apesar de otiima, torna-se inviavel para $N>12$ (explosao $O(N!)$). O Greedy Ingenuo evidencia que 'ser guloso' sem a estrutura correta (heap + custo acumulado global) nao garante otimalidade — gap empirico de ate 231%.

N	FB Tempo (ms)	Dijkstra (ms)	FB Chamadas	Dijk Relax.	Gap %
3	0.021	0.037	4	6	0.00%
5	0.024	0.026	8	10	0.00%
8	0.057	0.035	29	22	0.00%
10	0.090	0.095	50	28	0.00%
12	0.060	0.043	31	32	0.00%
20	inviavel	0.097	—	56	—
50	inviavel	0.125	—	146	—
100	inviavel	0.442	—	298	—

6. Conclusao e Conexao com ODS

O sistema demonstrou que a combinacao de estruturas de dados eficientes (BST + heapq + dicionario) com o algoritmo de Dijkstra resolve o problema de triagem e roteamento de emergencia de forma otima e escalavel. Para o Cenario A (RS), a rota de menor custo de Porto Alegre ate Mucum (risco critico 0.95) e de 2.0h via Montenegro-Lajeado, identificada em menos de 0.1ms. Para o Cenario B (MATOPIBA), a agenda a partir de Palmas (TO) prioriza corretamente Palmas de Monte Alto (BA, risco 0.91) como primeiro destino. A BST garante que a consulta de municipios criticos custa $O(k+\log N)$, independente do tamanho da rede.

ODS	Conexao com o sistema
ODS 2 — Fome zero	Rota minima ate cooperativas agricolas afetadas pela seca/enchente
ODS 9 — Infraestrutura	MST (Prim) define a rede de menor custo para logistica de emergencia
ODS 11 — Cidades	BST e Dijkstra permitem triagem e resposta rapida em cidades resilientes
ODS 13 — Clima	Sistema alimentado por dados de satellite (Sentinel, GOES-16, MODIS)

7. Referencias

- CORMEN, T. et al. *Introduction to Algorithms*, 4a ed. MIT Press, 2022. Caps. 22-25.
- SEDGEWICK, R.; WAYNE, K. *Algorithms*, 4a ed. Addison-Wesley, 2011. Parte 4: Grafos.
- SKIENA, S. *The Algorithm Design Manual*, 3a ed. Springer, 2020.
- DEFESA CIVIL RS. Boletins de Enchentes 2024. <https://www.defesacivil.rs.gov.br>
- NASA EARTHDATA. MODIS NDVI MYD13A3 2023. <https://earthdata.nasa.gov>
- INMET. Banco de Dados Meteorologicos (BDMEP) 2023. <https://bdmep.inmet.gov.br>
- IBGE. Malha Municipal e Dados Socioeconomicos 2024. <https://ibge.gov.br/geociencias>
- DNIT. Malha Viaria Federal RS. <https://dnit.gov.br>
- Carta Internacional Space and Major Disasters. <https://disasterscharter.org>